

Panorama completo do JavaScript Básico

Leitura organizada do que foi aprendido nos códigos do zip

1. Visão geral

No pacote enviado existem 20 arquivos .js distribuídos em pastas de fundamentos. O material cobre variáveis, tipos de dados, conversão, operadores, precedência, incremento/decremento, Math, condicionais, switch, tratamento de exceções, laços de repetição e funções.

Também aparecem exercícios em funções/desafios, o que mostra uma evolução de teoria para prática. Há alguns nomes desalinhados ou com erro de digitação, mas o conteúdo em si está bem encaminhado.

2. Organização recomendada para revisão

A ideia aqui é deixar tudo em uma ordem de estudo que faça sentido e seja fácil de revisar:

Bloco	Arquivos atuais	O que revisar primeiro
1. Fundamentos	variaveis, tipos de dados, conversao, expressoes-operadores, incremento-Decremento, ordem-de-precedência, mathss	Como o JavaScript guarda dados e como faz contas
2. Controle de fluxo	condicionais, switch, tratamento-de-exceções	Como o código decide o que fazer em cada caso
3. Repetição	Estruturas-de-repetição/for.js, while.js	Como repetir tarefas e percorrer listas/objetos
4. Funções	funções/função.js, funcao-anonima.js, callbackFunction.js, calculadora.js, desafios	Como criar blocos reutilizáveis e resolver problemas

3. Lista completa do que foi aprendido

Variáveis e escopo

- Diferença entre var, let e const.
- var permite reaproveitar nomes e tem escopo mais amplo; let e const respeitam bloco.
- const não deve ser reatribuído depois de definido.
- Boas práticas de nomeação: nomes descritivos, sem acento, sem caracteres especiais e com case consistente.

Tipos de dados

- String para texto.
- Number para números inteiros e decimais.
- Boolean para true/false.
- undefined quando uma variável foi declarada, mas não recebeu valor.
- null como valor intencional de ausência.

Strings e exibição de texto

- Aspas simples, aspas duplas e crase.
- Uso de template literals para interpolar variáveis com `${...}`.
- Quebra de linha e formatação mais legível com crase.
- Concatenação de textos com `+`.

Conversão e coerção

- Conversão explícita com `Number()` e `toString()`.
- Coerção automática do JavaScript em expressões como `"10" + 5`.
- Uso de `typeof` para inspecionar tipos.

Operadores

- Operadores aritméticos: `+`, `-`, `/`, `*`, `**` e `%`.
- Operadores de comparação: `==`, `===`, `!=`, `!==`, `>`, `<`, `>=`, `<=`
- Operadores lógicos: `&&`, `||` e `!`.
- Uso prático de percentual/paridade com resto da divisão.
- Diferença entre comparação frouxa e estrita.

Precedência

- Ordem em que as operações são resolvidas.
- Agrupamento, chamadas, incrementos, operadores unários, exponenciação, multiplicação/divisão, soma/subtração, comparações, igualdade, `&&`, `||` e atribuição.

Incremento e decremento

- Uso de `++` e `--`.
- Diferença entre pós-incremento e pré-incremento.
- Atribuições compostas como `+=`.

Math

- `Math.abs`, `Math.ceil`, `Math.floor`, `Math.round`, `Math.trunc`.
- `Math.pow`, `Math.sqrt`, `Math.min`, `Math.max`, `Math.random`, `Math.PI` e `Math.sin`.

Condicionais

- `if`, `else if` e `else`.
- Condição ternária com `? :`.
- `Truthy` e `falsy`.
- Identificação de valores que viram falso em contexto booleano: `false`, `0`, `""`, `null`, `undefined` e `NaN`.
- Leitura do fluxo de decisão em notas e aprovação/reprovação.

Switch

- Estrutura `switch/case/default`.
- Uso de `break` para impedir que os casos continuem executando.
- Aplicação em menus e opções numeradas.

Tratamento de exceções

- `try`, `catch` e `finally`.
- Como capturar erros sem quebrar o programa.
- Uso de `throw` para gerar erro manualmente.

Repetição

- for com contador.
- for...of para percorrer valores de arrays e strings.
- for...in para percorrer propriedades de objetos.
- while para repetir enquanto a condição for verdadeira.

Funções

- Declaração de função tradicional.
- Parâmetros e argumentos.
- return para devolver resultado.
- Funções anônimas.
- Arrow functions.
- Funções que recebem outras funções como callback.
- Criação de uma calculadora com switch dentro da função.

Desafios práticos

- Contar de 1 até um número.
- Montar tabuada com validação entre 1 e 10.
- Exibir a hora exata.
- Exibir o ano atual.
- Comparar dois números e descobrir o maior.
- Problema da prefeitura com salário e filhos.
- Calcular taxa mensal de juros.
- Criar regra de desconto com várias condições.

4. Revisão arquivo por arquivo

Arquivo	O que ele ensina	Observações importantes
variaveis/variaveis.js	var, let, const, escopo e boas práticas de nomes	Mostra a base de declaração de variáveis e reforça que nomes bons ajudam a leitura.
tipos de dados/index.js	Strings, numbers, booleans, undefined, null, concatenação e template literals	É um arquivo de introdução aos tipos primitivos e à diferença entre texto e número.
tipos de dados/conversao.js	Conversão explícita e coerção automática	Mostra como transformar string em número e como o JavaScript faz conversões sozinho.
expressoes-operadores/index.js	Operações aritméticas e resto da divisão	Apesar do nome genérico, o foco aqui é matemática básica no código.
incremento-Decremento/index.js	++, --, += e diferença entre pós/pré-incremento	Ótimo para entender mudança de valor em uma variável ao longo do tempo.
ordem-de-precedencia/index.js	Prioridade das operações	É praticamente uma cola/resumo teórico para não errar contas e expressões.
mathss/math.js	Principais métodos da biblioteca Math	É um guia de consulta rápida; quase todo o conteúdo está em comentário.
operadores-logicos/index.js	Comparações ==, ===, !=, !==, >, <, >=, <=	Explica bem por que o operador estrito é mais seguro.

operadores-logicos2/index.js	&&, e !	Apresenta lógica booleana aplicada em condições.
condicionais/index.js	Ternário e valores truthy/falsy	Mostra uma forma curta de escrever decisão e os valores que o JS considera falsos.
condicionais/if,else,elseif.js	Fluxo com if, else if e else	Bom para memorizar a estrutura e a sintaxe correta.
condicionais/switch.js	Escolha por casos com break e default	Ideal para menus, números de opções e situações com vários caminhos fixos.
condicionais/tratamento-exceções/index.js	try/catch/finally e throw	Ensina a lidar com falhas e a encerrar o fluxo com segurança.
Estruturas-de-repetição/for.js	for, for...of e for...in	Mostra três jeitos diferentes de percorrer dados.
Estruturas-de-repetição/whilw.js	while	Arquivo simples para entender repetição com condição.
funções/função.js	Funções, parâmetros, argumentos e return	É um dos arquivos mais importantes da pasta; ensina a base de reutilização.
funções/funcao-anonima.js	Função anônima e arrow function	Introduz uma forma mais curta de escrever funções.
funções/callbackFunction.js	Callback	Mostra uma função sendo passada como argumento para outra.
funções/calculadora.js	Função com switch para operações matemáticas	Um mini projeto de calculadora com retorno de resultado.
funções/desafios/index.js	Exercícios de lógica e uma solução completa de desconto	Tem vários exercícios comentados e um desafio implementado no final.

5. Pontos de atenção e limpeza de organização

- Existe um padrão de nomes com pequenos erros de digitação: `whilw.js`, `mathss`, `expressoes-operadores`, `tratamento-exceções` etc. Vale padronizar tudo depois.
- Alguns exercícios estão somente comentados; isso é normal para estudo, mas para revisão rápida é melhor separar `teoria`, `exercícios` e `soluções`.
- O arquivo `funções/desafios/index.js` ficou como um bloco de rascunho + solução final. Para estudar, ele funciona melhor se for dividido por exercício.
- O arquivo de `for.js` tem um detalhe comum de iniciantes: ao usar índice de array, a condição costuma ser `i < students.length`, não `<=`.
- O arquivo `math.js` é excelente como resumo visual, mas poderia ganhar exemplos executáveis para fixar melhor.

6. Ordem de revisão ideal

- 1) variaveis
- 2) tipos de dados
- 3) conversao
- 4) expressoes-operadores
- 5) operadores-logicos e operadores-logicos2
- 6) incremento-Decremento

7. 7) ordem-de-precedência
8. 8) math
9. 9) condicionais (if/else, ternário, truthy/falsy, switch)
10. 10) tratamento-de-exceções
11. 11) estruturas de repetição
12. 12) funções
13. 13) desafios práticos

7. Panorama final

No geral, você já aprendeu a base real de JavaScript: criar variáveis, reconhecer tipos, converter valores, operar com números, comparar dados, tomar decisões, repetir ações e encapsular lógica em funções. Também já tocou em tratamento de erro e em exercícios práticos mais próximos de problemas reais.

O próximo passo natural é deixar os arquivos mais limpos, separar exercícios resolvidos dos rascunhos e transformar cada tópico em uma revisão curta + prática.